Moderation Flow

Overview

Moderation in Yenkasa spans:

- post approval before publication
- user and post reports
- global post deletion
- user suspension and blocking
- community review workflows

The current design mixes dedicated moderation routes with approval-specific routes and role-gated community review flows.

Current implementation

Post approval

- `routes/postapproval.routes.js`
- reviewer roles: moderator, admin, junior developer, senior developer
- `GET /pending` also backfills missing approval rows and sends pending notifications to approvers
- approval updates post status to `approved`, rewards the creator, emits feed events, and triggers community notifications

Moderation actions

- `routes/moderation.routes.js`
- pending moderation item listing
- approve/reject moderation items
- delete posts globally
- suspend or globally block users
- user reporting endpoints

Community review

- `routes/community.routes.js`
- pending communities are reviewable by approver roles

Important modules

- `routes/postapproval.routes.js`
- `routes/moderation.routes.js`
- `models/ModerationItem.model.js`
- `models/postapproval.model.js`

Known issues

- `GET /post-approval/pending` performs side effects and backfill work
- moderation authority is split across different role utilities
- moderation actions are broad and mostly synchronous

Scaling concerns

- notifying approvers from the pending read path can amplify traffic
- moderation item queries rely heavily on populate and broad scans

Recommended improvements

1. split read-only pending review endpoints from notification/backfill tasks
2. unify moderation authority behind one RBAC service

3. add audit trails for all destructive moderation actions
4. introduce moderation queues for heavier review workflows